

JVMud User Manual

JVMud Project

Version 0.1.0-SNAPSHOT

Table of Contents

How to use this manual	1
Getting Oriented	2
Installing And Running	6
Playing And Testing	12
Operating JVMud	13
LPC Authoring	15
Mudlibs	27
Developer Reference	28
Troubleshooting	30
Appendixes	32

How to use this manual

JVMud is an LPC/LPMud engine for the JVM. This manual explains how to understand its design, run a local world, inspect LPC objects, and develop mudlib content against the current repository.

The documented version is 0.1.0-SNAPSHOT. The platform provides a compiler, world model, hosted instances, Telnet sessions, storage adapters, and an admin shell. Compatibility is incremental; a successful build or preload scan does not prove complete gameplay or production readiness.

- **Evaluating JVMud:** read *Getting Oriented* for philosophy, architecture, and the core package map.
- **Running or playing a local world:** follow *Installing And Running* and the *Small Mercies walkthrough*, then *Playing And Testing*. Commands in terminal blocks run from the repository root.
- **Operating or debugging:** use *Operating JVMud* and *Troubleshooting*.
- **Writing LPC:** follow *LPC Authoring* and *Mudlibs*; keep the configuration and command references nearby.
- **Changing Java:** start with the package map, then *Developer Reference*.

Examples identify whether input belongs in a terminal, the admin shell, or a Telnet client. Replace angle-bracket placeholders with actual values; do not type the angle brackets. A mudlib is the LPC source and configuration that define a game.

The manual owns procedures. The website's Principles and Glossary pages explain concepts; its efun and lifecycle pages own focused boundary references. Java API pages describe Java contracts, not player commands.

Terms used in procedures

A **Player** is an identity; a **Session** is a connection; a **Persona** is the Entity through which that Player acts in a World. A **Place** provides locality, a **Link** connects Places, and a **Location** identifies containment. A loaded LPC object can also be a service with no physical presence.

A **manifest** is the profile's `.config` file. It selects source paths, startup objects, lifecycle mappings, language features, and engine capabilities. A **profile** includes this manifest and the mudlib-side adapters needed to host that content. Legacy terms such as `room` and `heart_beat` are used when naming actual LPC paths or compatibility methods.

Getting Oriented

Start with what JVMud means by a world, then follow how its packages collaborate. The architecture is an ownership model; it is not a claim that every planned engine behavior is complete.

Overview

JVMud compiles LPC source to JVM bytecode and loads that code as live objects. The engine models shared world concepts; a mudlib supplies fiction, commands, rules, and presentation. The hosted instance brings these parts together, while Telnet gives players a line-oriented way to participate.

The distribution includes Small Mercies, a tiny teaching world. Start with its walkthrough, or choose or author a compatible LPC source tree and declare its hosting requirements in a manifest. Shared launchers always require that manifest explicitly.

The Java code is one Maven project with responsibility-based packages. The packages described here are not separately released Maven modules.

The JVMud Philosophy Of MUD

JVMud is built around eight core concepts:

- Game
- Text
- Interactive
- Multiplayer
- World
- Persistence
- Temporality
- Presence

Together, these concepts define what JVMud means by MUD.

A MUD is an interactive, multiplayer, text-only game set in a persistent, temporal, player-present world.

JVMud is not a general-purpose simulation framework, chat server, collaboration platform, generic game engine, or generic MUD framework. It is specifically and by design an LPMud engine: a platform for imagining, creating, operating, and playing LPC-authored game worlds in the tradition of LP MUDs.

Game

The world exists to be played. JVMud assumes rules, mechanics, goals, challenges, progression, or other game-like structures.

Text

Text is the primary medium through which the world is perceived and manipulated. It is a first-class part of the engine's design, not a skin over a visual game.

Interactive

Players can affect the world, and the world can affect players. Actions have consequences.

Multiplayer

Multiple players inhabit the same world simultaneously. The world is shared.

World

The game takes place within a concrete virtual domain. JVMud describes this domain in terms of Places, Links, Entities, and Movement.

Places give the world locality. Links make Places traversable. Entities are the things that exist in that structure. Movement changes where an Entity is present.

Persistence

The world and its state endure independently of any individual player's session. A player may disconnect and later return to a world whose state has continued to exist.

Persistence does not require eternal continuity. Reboots, resets, and selected save/restore behavior can all be compatible with a persistent world.

Temporality

The world can change through time. Events occur, actions have consequences, and state evolves.

Temporality is separate from Persistence: a world can have active time, saved state, restored state, and reset state without treating all of those as the same idea.

Presence

Presence means that a player experiences the game from within the World. The player is not an outside observer manipulating a simulation from above.

In JVMud terms, a player participates through a Persona: an Entity that serves as the player's current point of perception and action. A Persona is located somewhere, and that location determines what the player can perceive and what actions are locally available.

Players are somewhere, not everywhere. Presence creates locality of information, which makes exploration, discovery, concealment, surprise, navigation, and mystery possible.

LPC Compatibility And Engine Vocabulary

JVMud has one mudlib and language target: LPC for LPMud-style worlds. Its compatibility work exists to make JVMud a deeper LPMud platform, not to broaden it into a multi-language MUD framework.

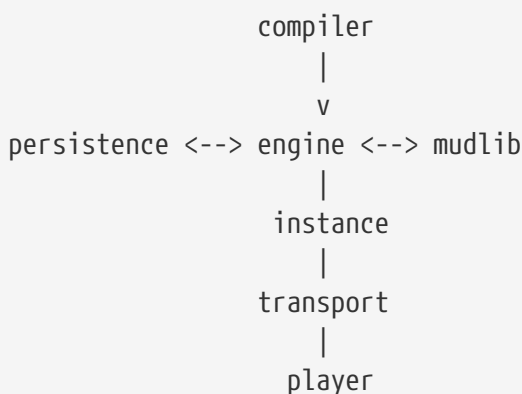
An LPMud, in JVMud's sense, is a game world authored in LPC, compiled into live game objects, and able to be rewritten, recompiled, and reloaded while the game continues running.

That does not mean JVMud adopts every legacy LPMud driver term as engine ontology.

Legacy names such as rooms, objects, heartbeats, applies, call_outs, and master objects may appear in compatibility surfaces, mudlib source, and LPC-facing documentation. The engine-facing model remains JVMud's own model: Game, Text, Interactive, Multiplayer, World, Persistence, Temporality, and Presence, built from Places, Links, Entities, Players, Sessions, and Personas.

Architecture

JVMud is organized around a small set of architectural responsibilities. The diagram is the responsibility map for the Java package layout and for the boundary between JVMud-owned code and mudlib-authored content.



The engine is the center of the architecture. It owns JVMud truth: the world model, canonical state machinery, and rules that should still make sense in a non-Telnet, non-LPC, test-only JVMud world.

That includes Places, Entities, Locations, Links, Personas, Sessions, world time, mudlib boundary events, and the `WorldRuntime` that enforces engine rules such as single containment and valid movement.

The compiler feeds the engine by turning LPC source into executable JVM code. Compiler packages own preprocessing, scanning, parsing, semantic analysis, IR, bytecode generation, `efun` contracts, generated-code helpers, and LPC runtime loading. Compiler helper classes support generated bytecode; they do not define the engine ontology.

The mudlib is the authored world. It supplies LPC content, fiction, object behavior, commands, rules, and compatibility policy. Mudlibs are deliberately outside the JVMud Java package tree and live under `mudlibs/`, because they are content hosted by JVMud rather than JVMud engine code.

Persistence sits beside the engine because durable state is a cross-cutting storage concern, not an engine ontology concern. Filesystem and JDBC adapters belong in persistence packages so the engine and instance layers can ask for durable services without embedding file formats or database details.

The instance layer is one live hosted realization of the architecture. It boots a mudlib, assembles compiler output with engine state, owns the active `WorldRuntime` object for that hosted world, attaches Personas, dispatches mudlib lifecycle hooks, and routes mounted mudlib worlds through the `MudlibRouter`. The instance is concrete in a way the engine is not: it knows which mudlib is booted, which LPC objects are loaded, and which players are currently attached.

The transport layer is the human connection boundary. Telnet code owns sockets, sessions, echo behavior, line input, output delivery, and connection lifecycle. Transport turns a human connection into input for an instance; it does not own world rules.

The player is at the bottom because all of this machinery exists to create an interactive, player-present text world. A player reaches JVMud through a transport, enters a running instance, experiences engine-owned world reality through a Persona, and interacts with mudlib-authored behavior compiled into JVM code.

As a rule of thumb: if a concept defines what JVMud means by a world, it belongs near engine. If it exists because a particular hosted mudlib is running, it belongs near instance. If it speaks a connection protocol, it belongs near transport. If it stores durable state, it belongs near persistence. If it is authored LPC content, it belongs in a mudlib.

Core packages and repository tour

All Java package names below begin with `io.github.protasm.jvmud`.

Package	Responsibility and starting points
engine	World rules and identity. Start with <code>world.WorldRuntime</code> , <code>identity</code> , <code>time.WorldScheduler</code> , and <code>mudlib.MudlibBoundary</code> .
compiler	LPC compilation and execution. Start with <code>JVMudCompiler</code> , <code>pipeline</code> , and <code>exec.LPCRuntime</code> . <code>runtime</code> supports generated bytecode.

Package	Responsibility and starting points
instance	Boot and hosted-world orchestration. Start with <code>MudlibBoot</code> , <code>MudInstance</code> , and <code>MudlibRouter</code> .
transport	Player connection mechanics. Start with <code>telnet.TelnetServer</code> .
persistence	Durable adapters under <code>filesystem</code> and <code>jdbc</code> .
cli	Local object sandbox. Start with <code>AdminCli</code> .

`src/main/java/` contains these packages; `src/test/java/` mirrors them. `mudlibs/<name>/source/` contains LPC content, and `mudlibs/<name>/jvmud/` contains manifests, adapters, and profile-specific scripts. Inspect the selected profile before assuming all mudlibs have the same storage layout.

`manual/` contains editable AsciiDoc; `docs/` contains the site, generated manual, and API. Shared tools live in `scripts/`. `pom.xml` defines dependencies and build profiles. See [Documentation Generation](#) before editing generated pages.

Installing And Running

This chapter gets one local listener running and proves that a player can reach it. Database requirements depend on the selected profile.

Installing

You need a JDK that can compile Java 21, Maven, Git, and a POSIX shell for the repository launch scripts. Some profile-owned smoke scripts also require Bash or Python 3. A Telnet-capable client is needed for interactive play. The bundled Small Mercies walkthrough needs no database or saved-account setup.

```
git clone https://github.com/protasm/jvmud.git
cd jvmud
java -version
mvn -version
mvn -DskipTests compile
```

Check that Maven reports the intended JDK. The compiler release is set to 21 in `pom.xml`. The first build needs access to Maven dependencies; later builds may use the local cache. Successful compilation creates `target/classes/`.

For an existing checkout, run commands from its root. Do not clone over local work. Tests are described in [Testing And Verification](#); compiling alone does not run them.

Running JVMud

For a ready-to-run example, use [First steps with Small Mercies](#). To host your own content, select a JVMud-compatible LPC source tree and its manifest. JVMud is the host; it does not choose game content implicitly. The manifest must describe real source files, including an initial Place and the profile's player behavior. Small Mercies is the bundled teaching example; the same host contract applies to other compatible content.

In a terminal at the repository root:

```
scripts/jvmud-start <mudlib-config-file>
```

Replace `<mudlib-config-file>` with the absolute path to your manifest, or a path relative to the repository root. Do not type the angle brackets. The script compiles Java, resolves the dependency classpath, and runs the Telnet server in the foreground. Wait for the preload summary followed by `JVMud mudlib listening on 127.0.0.1:4000` before connecting.

The launcher defaults to loopback port 4000. Use `--bind 0.0.0.0 --port 4000` to accept incoming connections on all IPv4 interfaces, or choose a specific local address and a different port. Clients use the server's actual address; allow the selected TCP port through applicable firewalls and router forwarding. Each listener needs an available address/port pair. Mounted profiles, if declared, must also be present and valid. See [Configuration Reference](#) for manifest keys.

To capture detailed startup failures:

```
scripts/jvmud-start --trace-startup-loads <mudlib-config-file> > target/startup.log  
2>&1
```

This still runs in the foreground. Inspect the log from another terminal and stop with Ctrl+C. Trace output can be large; begin with the preload summary and first failure. To see launch syntax, use `scripts/jvmud-start --help`.

Ctrl+C runs the server's shutdown path. Persistence is profile-specific; shutting down does not promise a snapshot of every loaded object's state. Add `--admin-port 4100` to enable the authenticated local admin listener, then connect with `scripts/jvmud-cli --port 4100`. The CLI controls this live primary world; its port is separate from the player port.

First steps with Small Mercies

Small Mercies is JVMud's bundled toy mudlib: five rooms, a few talkative NPCs, and sparring opponents with more confidence than good sense. It demonstrates a native LPC profile without database setup, account registration, or saved characters. Use it to learn the player flow and to find small source examples.

Launch and create a guest

From the repository root:

```
scripts/jvmud-start mudlibs/smallmercies/jvmud/smallmercies.config
```

Wait for JVMud mudlib listening on 127.0.0.1:4000. In a second terminal or Telnet-capable MUD client, connect to 127.0.0.1 on port 4000:

```
telnet 127.0.0.1 4000
```

Answer each prompt separately:

1. Choose a name containing 2–16 letters. It must be different from the names of players currently connected.
2. Choose `male` or `female`.
3. Choose `warrior` or `mage`.

You arrive in the **Village Square**. Type `help`, then `score`. There is no password. A guest character exists for one connection: disconnecting ends that character's visit, and reconnecting starts character creation again.

Class	STR	INT	DEX	Starting HP
Warrior	12	6	9	32
Mage	6	12	9	26

Warriors use STR for their attacks; mages use INT for sparks. DEX helps dodge. `score` also shows victories and the current opponent, when fighting. Gender changes the character description, not the attributes or rules.

Explore the whole world

Every exit has a return route. Read `look` to see a description, exits, and nearby characters. Use full directions, their initials, or go `<direction>`.

Place	Exits
Village Square	west to The Resting Hero; north to the Municipal Herb Garden; east to Very Short Lane.
The Resting Hero	east to the square.
Municipal Herb Garden	south to the square.
Very Short Lane	west to the square; down to the Cellar of Mild Peril.
Cellar of Mild Peril	up to the lane.

From a fresh arrival in the square, enter these player commands one at a time:

```
look
talk mayor
west
talk innkeeper
rest
east
north
look goose
south
east
down
look rat
up
west
```

You finish back in the square. `rest` restores full health only at the inn. `talk` addresses an NPC in your current Place. Try `who` to list connected adventurers. `say Hello!` and `smile`, `wave`, `bow`, or `laugh` provide local communication. Connect a second guest to try these with another player.

Try a short sparring match

From the square, go `north`, then type `attack goose`. Combat repeats every two seconds; you do not need to repeat the attack command. `kill goose` is an alias. Use `stop` or move south to end the bout. The rat in the cellar is the other sparring opponent; the mayor and innkeeper are friendly.

A defeated NPC rests for 20 seconds before recovering. A player defeat sends you to the inn with 1 HP; `rest` restores full health. NPCs spar with one challenger at a time; their remaining HP persists when a visitor stops or leaves. There is no player-versus-player combat. If an opponent is resting or occupied, return later rather than repeatedly issuing `attack`.

Use `score` to check health and victories, then `quit` to end your visit. `//quit` is the default transport disconnect control. Neither creates a saved character. NPC state remains shared while the server runs; restarting resets the world. Stop the server with Ctrl+C in its terminal when finished.

Read the implementation

The manifest selects `room/square` and `player/adventurer`, maps object loading to `create`, interaction scope to `init`, session connection to `connect`, and session disconnection to `disconnect`. It grants `session_control` and maps recurring ticks to `tick`; there is no filesystem-account policy or database.

Path under <code>mudlibs/smallmercies/</code>	What to learn
<code>jvmud/smallmercies.config</code>	Explicit source root, preloads, Persona, initial Place, lifecycle mappings, and capability grant.

Path under <code>mudlibs/smallmercies/</code>	What to learn
<code>source/lib/room.c</code>	Shared descriptions, authored exit lookup, and NPC creation.
<code>source/room/*.c</code>	Small Place definitions with prose, exits, and resident NPCs.
<code>source/player/adventurer.c</code>	Guest creation, command registration, movement, communication, score, and recurring combat.
<code>source/npc/villager.c</code>	NPC identity, dialogue, single-challenger combat, and deferred recovery.

These LPC examples use JVMud-native efuncs. Game rules stay in the mudlib; the host supplies execution, session, location, and scheduling services. Read the source before changing behavior, then replay the relevant route after restarting. Use a separate copy for experiments you do not intend to keep in the distribution.

The two-guest live Telnet check lives at `src/test/scripts/smallmercies/smoke-play.py`.

Connecting With Telnet

In a second terminal, use a Telnet client:

```
telnet 127.0.0.1 4000
```

If your system has no `telnet` command, use a Telnet-capable MUD client with host `127.0.0.1` and port `4000`. Follow the mudlib's login prompts, then try `help` and `look`. Use `//quit` to disconnect from profiles using the default transport prefix. The listener keeps running until stopped in its own terminal.

Optional GMCP integration

JVMud's Telnet listener supports ordinary line-oriented clients and negotiates the Generic Mud Communication Protocol (GMCP) as Telnet option 201. The server offers GMCP with `IAC WILL GMCP`; clients that accept it can exchange UTF-8 GMCP subnegotiations without affecting the visible text stream.

Mudlibs with the `session_control` engine capability can use:

- `jvmud_gmcp_enabled()` or `jvmud_gmcp_enabled(user)` to test negotiation state.
- `jvmud_send_gmcp(package)` for a package without a payload.
- `jvmud_send_gmcp(package, payload)` to JSON-encode LPC strings, numbers, arrays, and string-keyed mappings.

An interactive LPC object may implement `client_protocol_changed(string protocol, int enabled)`

to react to negotiation and `receive_gmcp(string package, mixed payload)` to receive decoded client messages. JVMud owns Telnet framing and JSON conversion; mudlib code owns the meaning of packages such as `Room.Info` and `Char.Vitals`.

Smoke Tests

For the bundled toy, run its focused integration test from the repository root:

```
mvn -Dtest=SmallMerciesTest test
```

This boots the real profile and exercises character creation, movement, room-local communication, and scheduled combat. It does not start a TCP listener; use the route in [First steps with Small Mercies](#) for live transport verification.

For an automated live check, start a fresh Small Mercies listener as described in the walkthrough. Keep it running, then use a second terminal:

```
python3 src/test/scripts/smallmercies/smoke-play.py
```

The script connects to loopback port 4000, creates two disposable guests, and checks login, both classes, chat, an emote, the tutorial route, combat, score, rest, and quit. Success prints `PASS: live guest login...`. It does not launch or stop the server. Use a dedicated development listener so another player's sparring match or a resting NPC does not interfere.

The shared preload-scan script expects `preload_file`; Small Mercies declares `preload_objects` inline. Use the focused test rather than that scanner for this profile. A host smoke test should prove the complete route from launch to interaction, using the profile you intend to run:

1. Start the listener with its explicit manifest and wait for the listening message.
2. Connect a Telnet client and complete the configured login or entry flow.
3. Exercise a known command and verify its expected text.
4. Move through a known Link and verify the new Place.
5. Disconnect and reconnect to check cleanup and any state the profile promises to save.
6. Stop the listener and retain the first error if a step fails.

Use a disposable account and state directory selected for the test. Profile-owned scripts may automate these steps, but the shared engine does not prescribe game names, account names, commands, locations, or credentials.

Passing proves that profile's particular transcript. It does not prove all language behavior or multi-user reliability. Separate compiler assertions, preload scans, and live player evidence when reporting results.

Playing And Testing

Player commands belong to the selected mudlib. JVMud handles transport controls and delivers ordinary input to the hosted world.

Login Flow

Connect only after the listener reports that it is ready. Follow the profile's prompts rather than sending gameplay commands before entry is complete.

There are two configuration patterns:

- With `session_policy = filesystem_accounts`, JVMud performs local account-file and password-verification work. It then dispatches `player_persona_resolved` and `player_entered_world` to mudlib-owned methods selected in the manifest.
- Without a managed session policy, the configured interactive object supplies the login conversation through its mapped session-connection lifecycle method.

`persona_object` selects the interactive LPC source; `player_object` is the legacy manifest spelling. Method names and prompts belong to that source tree, not to a universal JVMud game flow.

For a repeatable test, record the manifest, whether the account is new or existing, the prompt where input was sent, and the first unexpected reply. Exclude passwords and private account fields from shared transcripts.

Basic Commands

Read the selected world's welcome text and help after entry. A profile may provide verbs such as `help`, `look`, or movement directions, but JVMud does not require a universal gameplay vocabulary. Use the exact verbs and exits documented by your authored content.

For a first route, inspect your current Place, choose one listed exit, move, and verify that the description changes. Record the expected text in your test. These are player-client inputs; the local admin shell has a separate grammar.

With the default transport prefix, `//help` shows connection controls and `//quit` disconnects the client. A mudlib's own exit command can run game-specific saving and cleanup. For persistence tests, exercise that command as well as transport disconnect/reconnect.

Command Routing

Input passes from the Telnet session to the hosted instance and the profile's LPC command behavior. The configured `transport.control_prefix` reserves an escaped namespace for connection controls; its default is `//`.

- `//help` and `//quit` are transport controls in profiles using the default transport prefix.
- `look` and movement verbs go to the `mudlib` after login.
- Ordinary single-slash commands remain available to `mudlib` code.
- `load`, `inspect`, and `reload` run through the authenticated admin connection against the live primary world. They are not player Telnet commands.

If a command is unrecognized, first identify which of these contexts received it. Then check the active profile's lifecycle mappings and command registration.

Transcript Regressions

For a useful regression report, record the repository revision, manifest path, startup summary, client input, expected result, actual output, and server error. Use a small route with known starting state. A successful login is a different assertion from successful movement, persistence, or transfer between worlds.

Start with the Small Mercies walkthrough and the relevant tests under `src/test/java/io/github/protasm/jvmud/instance/` and `transport/`. Add a focused assertion for the first wrong behavior, then replay the route against a restarted listener when runtime code changes. Avoid treating a long transcript with no assertions as proof of compatibility.

Operating JVMud

Use the local admin shell to investigate object behavior, the manifest to define hosting policy, and the server log to diagnose a running listener.

Live Admin CLI

Start a server with an explicit, separate administration port:

```
scripts/jvmud-start --port 4000 --admin-port 4100
mudlibs/smallmercies/jvmud/smallmercies.config
```

Connect from another terminal as the same OS user:

```
scripts/jvmud-cli --port 4100
```

The CLI displays the selected server's admin port and world path. Commands act on that server's primary world, sharing its actual objects and execution lock with player input and ticks. No new world is booted. Mounted worlds are not selected by this interface. Every input line is an admin command, not a player command.

```
help
pwd
ls /
cat /room/square.c
objects
inspect room/square
call room/square short
quit
```

`/` is the manifest-selected mudlib source root. `objects` refreshes the live registry, including connected player objects. `quit` or terminal EOF disconnects only this client. The attached CLI rejects `boot`; select another server by connecting to its admin port. Stop the server in its own terminal with `Ctrl+C`.

Administration is opt-in and always binds `127.0.0.1`, even when player Telnet binds all network interfaces. A random credential is created with owner-only permissions at `~/.jvmud/admin/<admin-port>.token`. The client reads it automatically; normal server shutdown removes it. A stale credential after a crash requires explicit removal after confirming that the old server has stopped.

Override the credential path with `--admin-token-file <path>` on the server and `--token-file <path>` on the CLI. Startup never overwrites an existing credential. Use a different player/admin port pair for every independently running server.

To investigate a changed object, edit its LPC source, run `reload <path>`, and inspect the result. `clone <path>` creates an instance; `destruct <handle>` removes one. These commands affect live state. Reload does not migrate all state or references; verify callers and avoid assuming occupied rooms or players can be replaced transparently. Long administrative commands can delay player input and ticks because commands execute under the world's lock.

The complete command syntax is in [Commands](#). Compilation diagnostics use the running server's configured observer; the attached client does not replace it.

Runtime Model

`WorldRuntime` enforces single containment and rejects containment cycles. Places connect through Links; Entities can be contained by Places or Entities. Players participate through Personas attached to Sessions. Time-dependent work uses the engine scheduler.

Hosted LPC execution also keeps object environment and inventory state in its runtime context. Do not assume every loaded service object is a world-present Entity, or that all compatibility bookkeeping has been unified with `WorldRuntime`. The repository's `docs/DEFERRED_WORK.md` records this boundary and the remaining world-event/concurrency work.

The `filesystem_accounts` policy stores account records under `accounts/` in its selected mudlib root. LPC object save/restore and JDBC data are separate storage surfaces. Before an upgrade or reset, identify the active profile's state paths and database, stop writers, and preserve those stores. The project does not yet provide a universal backup/restore command or a verified production recovery

procedure; test restoration on a disposable copy of the profile's data.

Mudlib Boundary

The manifest declares how a profile participates in JVMud. `initial_place` selects the starting Place. `persona_object` (or `legacy_player_object`) selects the interactive LPC object. Preloads, include paths, lifecycle methods, syntax features, and capabilities are explicit.

To adapt a profile:

1. Begin with its current `.config` file and identify `mudlib_root`.
2. Verify the initial Place, Persona, and preload paths exist.
3. Map JVMud lifecycle events to the mudlib's method names.
4. Enable only the syntax and engine capabilities the profile needs.
5. Launch it, inspect the preload summary, and test login and a small command route.

Unknown keys fail validation. `language_features` controls optional syntax; `engine_capabilities` authorizes capability-gated efuns. Neither is a substitute for the other. See [Configuration Reference](#) for the supported keys.

`docs/ENGINE_MUDLIB_CONTRACT.md` is the detailed design contract. Keep native world semantics in the engine and profile vocabulary in manifests or dedicated shims.

Example declaration

The following is a manifest shape, not a complete runnable world. Each source path must exist in your authored tree; the chosen method names must match it.

```
game_id = <your-world-id>
mudlib_root = ../source
initial_place = place/start
persona_object = persona/player
lifecycle.player_session_connected = connect
transport.control_prefix = //
```

Add preloads, mapped world-entry behavior, account policy, syntax gates, and host capabilities only as required by that content. Validate startup before testing player interaction.

LPC Authoring

The [Small Mercies source map](#) links runnable examples of rooms, commands, NPCs, and scheduling. Start with a typed object and test it through the CLI attached to a development server. Then connect its behavior to the selected mudlib's commands and lifecycle. The language overview

below preserves practical LPC examples; compatibility names such as `write` and `sizeof` depend on the active profile's mappings.

Supported LPC

JVMud supports LPC as the source language for LPMud-style game worlds. The goal is not to preserve every historical driver quirk. The goal is to provide a clear, practical LPC platform for writing mudlib code that can be compiled, loaded, inspected, reloaded, and hosted by JVMud.

There is no single authoritative LPC. LPC grew through the work of hobbyists, enthusiasts, and mudlib authors building real worlds on real drivers. Over time, different drivers and mudlibs carried the language in different directions. Many LPC features are therefore best understood as dialect features, compatibility conventions, or driver-specific behavior rather than as parts of one universal standard.

When this manual says JVMud supports LPC, it means JVMud supports the LPC language surface described here, plus the compatibility behavior needed to host the mudlibs JVMud is designed to run.

This topic describes the supported language surface at a high level. It is a working authoring guide, not a complete grammar reference.

Source Files And Objects

An LPC source file compiles to a loadable game object. Source files may define:

- fields
- methods
- forward method declarations
- inherited parent objects

JVMud supports ordinary `inherit` declarations with string paths:

```
inherit "/obj/thing";
```

The parser also recognizes `virtual inherit` syntax for compatibility with mudlib source that uses it:

```
virtual inherit "/obj/base";
```

The runtime resolves object paths through the configured mudlib source root and include paths. Path policy is part of the runtime and mudlib configuration, not the LPC language itself.

Types

JVMud supports the LPC types currently needed by the compiler and hosted mudlibs:

- `int`
- `float`
- `string`
- `status`
- `object`
- `mapping`
- `mixed`
- `void`
- array forms such as `int *`, `string *`, and `mixed *`

Use explicit return types, field types, local variable types, and parameter types in JVMud-maintained LPC. Explicit types make compiler errors clearer and avoid importing legacy driver ambiguity into the engine contract.

```
string short() {  
    return "a brass lantern";  
}  
  
int count;  
  
void set_count(int value) {  
    count = value;  
}
```

Untyped methods and untyped parameters are rejected. Compatibility syntax does not remove the requirement for explicit signatures.

Declarations And Modifiers

Fields and methods may use the common LPC declaration modifiers that JVMud currently recognizes:

- `public`
- `private`
- `protected`
- `static`
- `nomask`
- `varargs`

- nosave
- deprecated

The `varargs` modifier applies to methods only and requires the `varargs` language feature. Protected evaluation, typed function literals, inline callables, and multi-value mappings also have explicit profile feature gates.

```
private int count;
nosave mixed transient_state;

public string query_name() {
    return "lantern";
}

varargs void message(string text, object target) {
    write(text);
}
```

Multiple field declarators are supported in a single declaration:

```
int x, y, z;
string *names, title_words;
```

Statements

JVMud supports the statement forms used by ordinary mudlib control flow:

- blocks
- expression statements
- if/else
- while
- do/while
- for
- foreach
- break
- continue
- return

Both `foreach (… in …)` and `foreach (… : …)` forms are supported. Mappings may be iterated with key and value variables.

```
int total(int *values) {
```

```

    int sum;

    foreach (int value in values) {
        sum += value;
    }

    return sum;
}

void describe_mapping(mapping values) {
    foreach (string key, mixed value in values) {
        write(key + ": " + value + "\n");
    }
}

```

Typed local declarations are supported in blocks and in for initializers. A for initializer may declare one typed local variable.

Expressions

JVMud supports the expression forms expected in practical LPC mudlib code:

- integer, float, string, true, false, and null-like zero values
- array literals
- mapping literals
- local and field access
- assignment and compound assignment
- method calls
- efun calls
- object-style invocation with $\rightarrow$
- parent calls through super
- indexing and slicing
- ternary expressions
- comma expressions in loop clauses
- protected evaluation with catch

Supported operator families include arithmetic, comparison, equality, logical, bitwise, shift, and unary operators.

```

mixed *values = ({ "north", "south", "east" });
mapping exits = ([ "north": "/room/green", "south": "/room/road" ]);

string first = values[0];

```

```
mixed *rest = values[1..2];

if (sizeof(values) > 0 && exits["north"]) {
    write("There is a way north.\n");
}
```

Adjacent string literals are accepted as one string value, which is useful for long room descriptions:

```
string long() {
    return "The corridor is narrow and cold. "
        "A brass lantern hangs from a hook.";
}
```

Collections

Arrays and mappings are first-class LPC values in JVMud.

Array literals use LPC's (`{ ... }`) form. Mapping literals use LPC's (`[ key: value ]`) form.

```
mixed *items = ({ "lamp", "key", "coin" });
mapping descriptions = ([
    "lamp": "A small brass lantern.",
    "key": "A tarnished iron key."
]);
```

Indexing is supported for arrays, mappings, and strings where the runtime type allows it. Slicing is supported for strings and array-like values.

Efun Calls

JVMud exposes engine functions as efuncs. Ordinary efun calls use the familiar LPC call form:

```
write("Hello.\n");
sizeof(({ 1, 2, 3 }));
```

Qualified `efun::name(...)` calls are supported for engine efuncs:

```
return efun::sizeof(({ 1, 2, 3 }));
```

The efun catalog is documented separately in this manual and in the generated Java API documentation for the engine efun registry.

Function References And Inline Callables

JVMud recognizes LPC function-reference forms used by mudlibs, including quoted function references and efun references:

```
#'db_close  
efun::db_handles()
```

Inline callable forms used with collection helpers such as `filter` and `map` are supported for the compatibility patterns currently exercised by hosted mudlibs.

Preprocessor And Includes

JVMud includes a preprocessor with support for ordinary include resolution, macro expansion, conditional directives, and source mapping. Include behavior depends on the runtime/compiler configuration for the mudlib being compiled.

Use includes for shared LPC constants and declarations, but keep engine-facing concepts in JVMud vocabulary. LPC headers should help mudlib code compile; they should not redefine the engine's model of Places, Entities, Players, Sessions, or Personas.

Runtime-Oriented LPC

JVMud supports the LPC patterns needed to host live LPMud objects:

- object loading
- method invocation
- inherited behavior
- selected lifecycle methods
- command hooks such as `init`, `add_action`, and `add_verb`
- player/session text output helpers
- save and restore helpers for LPC object state
- timed behavior through scheduler-backed compatibility functions

These features are compatibility surfaces over JVMud's engine model. They make LPC mudlib code usable without making legacy driver vocabulary the engine's own ontology.

Current Limits

JVMud's LPC support is intentionally practical and evolving. If a historical LPC feature is not needed by the hosted mudlibs, it may be absent, partial, or accepted only as a compatibility parse form.

In particular:

- supported LPC means the JVMud compiler and runtime currently have behavior for the construct, not that every legacy driver variant behaves identically
- compatibility syntax may exist before full runtime behavior exists
- JVMud-authored source should prefer explicit types and simple declarations
- engine concepts should be documented as JVMud concepts, not as legacy LPC driver concepts

When in doubt, prefer the forms already used by JVMud mudlib code and covered by compiler or telnet smoke tests.

Explicit Types

Declare return types and every parameter type. Untyped methods and parameters are compiler errors. Use `mixed` when a value intentionally varies, and `void` when a method returns no value.

These are the default typing rules. A mudlib can explicitly select `compiler.dynamic_types = true` to treat all declared value types as `mixed`. This does not allow missing method declarations; legacy declaration syntax requires its own opt-in. See the compatibility section for the policy boundaries.

For a disposable authoring exercise, save this as `example/counter.c` beneath the selected mudlib source root (create the directory if needed):

```
int count;

int add(int amount) {
    count += amount;
    return count;
}

int increment() {
    return add(2);
}

int query_count() {
    return count;
}
```

In the admin shell, run `load /example/counter`, then use its returned handle with `call <handle> increment` and `call <handle> query_count`. Both calls should return 2 on a fresh object. Use `inspect <handle>` to see its fields and methods. This exercise needs no legacy `efun` aliases. Remove only your disposable example when finished; keep upstream mudlib source intact.

The current admin `call` command passes supplied arguments as strings; it does not coerce numeric text to LPC integers. This exercise uses a no-argument method to call the typed integer helper. Test non-string argument contracts through LPC wrappers or the Java execution API.

Efuncs

Efuncs expose host operations to LPC. Ordinary calls may resolve through mudlib functions or the active boundary's translation table. `efun::name(...)` selects that efun surface; `jvmud::name(...)` requires an exact native JVMud efun name.

```
int sample_size() {
    return jvmud::jvmud_size(({ 1, 2, 3 }));
}
```

The native form does not invent a `jvmud_` prefix or translate `sizeof`. Legacy aliases belong in the profile. A recognized efun can still require an `engine_capabilities` grant, such as `mudlib_files`, `database`, `session_control`, or `host_control`.

Use [the efun reference](#) for signatures and categories, and `compiler.efun.builtin.CoreEfuncs` for implementation details. Do not duplicate the catalog in a mudlib guide. When a call fails, check its spelling, argument types, profile mapping, and required capability.

Ordinary JSON

`jvmud_parse_json(document)` converts an ordinary JSON document to LPC values. JSON objects become mappings with string keys; arrays retain their order; strings and numbers remain scalar values; booleans become status values 1 and 0; and JSON `null` becomes LPC false (0).

```
mapping item = jvmud_read_mudlib_json(
    "/data/items/i0001.json.gz");
```

`jvmud_read_mudlib_json(path)` is available to mudlibs with the `MUDLIB_FILES` engine capability. It parses either plain JSON or gzip-compressed JSON, detected from the file content rather than the filename.

`jvmud_format_json(value)` performs the inverse conversion for values that can be represented in ordinary JSON. Mapping keys must be strings. Because LPC false and a parsed JSON `null` share the value 0, formatting a parsed value is not promised to reproduce the original JSON token-for-token.

Bounded JSON Array Reads

Mudlibs with the `MUDLIB_FILES` engine capability may read a structural slice without materializing an entire JSON document:

```
mixed *items = jvmud_read_mudlib_json_array(
    "/data/catalog.json", "/items", 500, 100);
```

The second argument is an RFC 6901 JSON Pointer selecting an array. `offset` is the zero-based first

entry and count is the maximum number returned. JVMud scans forward through ordinary JSON, but skips unrelated branches and entries outside the requested range without constructing their LPC values. An empty pointer selects a root array. Plain and gzip-compressed documents are both accepted.

Lifecycle Hooks

JVMud emits native events; a manifest maps them to LPC methods. For example:

```
lifecycle.object_loaded = reset
lifecycle.interaction_scope_started = init
lifecycle.player_session_connected = logon
```

These are illustrative legacy method names, not required names. Native profiles can map the same events to their own typed LPC methods.

Read [the lifecycle reference](#) for each event's role. Verify the mapped method's signature in the profile and corresponding Java dispatch code before adding a hook. Test normal entry, failed input, and disconnect paths; recognizing a method name does not establish its invocation order or error behavior.

Perception

World communication is separate from private session output. An emitted perception reaches the source's location and its immediate contents, excluding the source and any explicitly excluded recipient. It does not recursively reach carried objects or apply sensory rules such as hearing distance or line of sight. Recipients are snapshotted before callbacks; destroyed recipients are skipped.

```
// Emit structured speech with text for connected observers.
jvmud_emit_world_event(this_object(), "speech", "mellon", "A voice says: mellon\n");
// Address one observer without broadcasting to its neighbors.
jvmud_deliver_world_event(target, "speech", "mellon", "A voice whispers: mellon\n");
```

These native functions take (mixed source_or_target, string kind, mixed content, string text). The existing `jvmud_emit_perceivable*` functions emit text events with their original message as content. `jvmud_write_to_lpc_object` remains private output and does not notify world observers.

Map `lifecycle.perception_received` to an observer method taking one mapping. Each observer receives its own mapping containing actor, source, location, kind, content, text, and directed (0 or 1). Missing object references are 0. The originating command actor is preserved during synchronous delivery; events outside a command may have no actor. The receiver is the current object. Text is delivered once to each connected recipient, independently of its hook.

A mudlib may also map `lifecycle.perception_delivery` to a boundary-object method with (object

observer, mapping event). It runs after the observer method and can adapt observations to the mudlib's conventions. LP245 uses this to call the optional `catch_tell` method with exact, unwrapped legacy text. Neither hook is required. Callbacks may emit further events; nesting beyond 64 deliveries raises an error and unwinds the delivery. Callback errors propagate to the existing command/scheduler error handling and can interrupt the remaining recipients.

Compatibility Shims

Keep imported source as upstream material. Put compatibility functions, aliases, headers, and lifecycle translations in the profile's `jvmud/` area where possible. `mfun_object` supplies mudlib-global helpers; `compatibility_object` and `compatibility_overrides` support explicit compatibility overrides.

For a missing legacy operation, first identify whether the gap is syntax, a native engine operation, or profile policy. Add generic compiler/engine behavior only when it belongs to JVMud; map legacy vocabulary in the profile. Do not relax explicit typing to accommodate an untyped legacy declaration.

Record the motivating source and test in `docs/COMPATIBILITY_FEATURE_PROVENANCE.md` when changing compatibility features. The result should remain usable without hardcoding a particular game into shared Java or launch scripts.

Opt-In Legacy Method Transpilation

A bridge manifest may explicitly set `transpilation.untyped_methods = true`. The default is false. JVMud's sibling `io.github.protasm.jvmud.transpiler` package adds explicit mixed return and parameter types to untyped method signatures in memory after preprocessing and before parsing. Includes, inherited objects, and global helper objects use the same profile setting. Existing explicit types and method bodies remain unchanged, and inserted tokens retain the original source locations. The compiler still rejects missing explicit method types when transpilation is disabled. This accommodation does not imply support for every legacy LPC dialect feature and never rewrites imported files.

Opt-In Dynamic Value Typing

Set `compiler.dynamic_types = true` when a mudlib's value declarations are documentary. The default is false. The compiler retains written type names but uses stable mixed storage for fields, locals, parameters, return values, and typed function literal declarations. Array declarations impose no restriction. Includes, inheritance, global helpers, and hosted bridge objects use the same policy. Assignments cannot narrow storage after compilation.

Modifiers, argument structure, native function contracts, explicit conversions, and runtime operation checks remain in effect. Duplicate methods of the same arity remain errors. Missing declarations require the separate untyped-method switch. Mixed defaults and empty returns use LPC zero. Existing unsupported syntax, including legacy casts, remains unsupported.

LP245 selects this policy instead of its former 51 field and 24 local type overrides. Its JSON file

retains only the method-varargs rule for `room/room.c:init`. Remove field/local type overrides before enabling dynamic typing; combining them is rejected. Checked overrides remain available for mudlibs using strict typing.

Restart the instance when changing typing policy so every object is recompiled under the same policy.

Checked Field-Type Overrides

Select a bridge-owned JSON file explicitly in the manifest:

```
transpilation.overrides = transpilation.json
```

The override file path is relative to the manifest directory. This setting is independent of `transpilation.untyped_methods`; neither enables the other.

```
{
  "field_type_overrides": [
    {
      "file": "obj/torch.c",
      "field": "amount_of_fuel",
      "expected_type": "string",
      "replacement_type": "int"
    }
  ]
}
```

`file` identifies a filesystem compilation unit relative to `mudlib_root`. Rules apply after preprocessing, so declarations expanded from its headers are part of that unit. They also apply when the unit is loaded as an inherited object, global helper, or bridge object. In-memory snippets without a source file path are not matched by file overrides.

The named field must occur exactly once with the expected type. A mismatch, missing field, or duplicate declaration stops compilation at the TRANSPILE stage. Invalid JSON, unknown keys, duplicate rules, missing source files, and unsupported field types are errors. Fields can use `int`, `float`, `string`, `object`, `mixed`, `mapping`, `function`, or `status`, with optional array stars. The JSON vocabulary also supports `local_type_overrides` and `method_varargs_overrides`; those adaptations are separate from field rules.

Grouped declarations are split when necessary to preserve neighboring fields' types, modifiers, and initializer order. Locals, parameters, method bodies, and string values are not field override targets. Source locations are retained for diagnostics and original files are never rewritten. Rules are read at manifest load; restart the instance to load changed configuration.

Mudlibs

A profile combines authored LPC with explicit hosting requirements. Native and legacy profiles share the JVMud host while owning their own vocabulary and game rules.

Layout

A useful source layout separates authored content, host configuration, and writable state. This is an illustrative convention, not a required game:

```
<content-root>/
  source/          LPC source, headers, and authored data
  jvmud/           manifest and host-facing adapters
  <state-area>/    profile-selected writable data
```

`mudlib_root` selects the actual source root. If the manifest lives in `jvmud/`, `mudlib_root = ../source` resolves from that directory. Inspect persistence code before choosing, moving, or cleaning state paths; not every store uses a sibling state directory.

Mounts use `mount.<game-id> = <config-path>`; the manifest explicitly names each mounted world. Paths resolve relative to the declaring manifest. Do not rely on sibling-directory discovery or any bundled game identity.

Small Mercies follows this separation under `mudlibs/smallmercies/`. Its guest characters are transient; it does not demonstrate durable account storage.

Validate a profile

Before hosting content, check these responsibilities against your source tree:

- the manifest's initial Place and Persona paths resolve to real LPC objects;
- includes, inherited objects, and preloads resolve under the intended root;
- lifecycle mappings name methods with the expected signatures;
- optional language features and engine capabilities match actual requirements;
- writable stores exist where the persistence policy expects them;
- mounted worlds boot independently before you exercise transfer.

For a load failure, reproduce the same object under the same manifest in the admin shell with verbosity `watch`. Changes to includes, syntax flags, or adapters can affect many objects, so rerun focused tests and the relevant live route.

Upstream Source Policy

Preserve upstream material and provenance. Prefer dedicated shims and manifest mappings to broad source rewrites. Keep necessary source fixes small and tested; a content bug is not automatically a compiler defect. Never commit private saves or secrets. Use `database.password_env` for database credentials.

Developer Reference

Use the core package map to choose where a change belongs, then follow the compiler and hosted-instance flows below. The [Java API](#) provides class and method contracts for all six core package families.

Compiler Pipeline

JVMud is easiest to understand as a collaboration between compiler, engine, and mudlib. The compiler turns LPC source into JVM classes. The engine owns the live world model. The mudlib supplies authored behavior and compatibility policy.

The compiler pipeline is deliberately staged so each step has a small job and a clear artifact:

Stage	Input	Output
Preprocess	LPC source text plus include paths and macro definitions	Expanded source with mappings back to the original files
Scan	Preprocessed source	A token stream
Parse	Tokens	An abstract syntax tree for one LPC object
Analyze	AST plus inherited object metadata and registered efun signatures	A semantic model with resolved names, signatures, and type information
Lower	Semantic model	Typed intermediate representation
Compile	Typed IR	JVM bytecode
Execute	Bytecode plus host runtime configuration	A loaded LPC object instance managed by the host-facing LPC runtime

The package layout follows that path. `preproc`, `scanner`, `token`, `parser`, `semantic`, `ir`, and `bytecode` are compiler stages. `pipeline` coordinates the stages and reports `CompilationProblem` values when a source unit cannot move forward. `efun` defines the LPC-facing function contract, and `efun.builtin.CoreEfuncs` provides JVMud’s built-in catalog.

The package `io.github.protasm.jvmud.compiler.runtime` is support code for compiled LPC bytecode: active execution context, truthiness, coercions, indexing, strings, arrays, and runtime type descriptors. It is not the JVMud engine ontology. Engine-owned concepts live under

`io.github.protasml.jvmud.engine` with `engine.world` for `WorldRuntime`, `Place`, and `Entity`; `engine.identity` for `Player`, `Session`, and `Persona` records; `engine.time` for scheduling; and `engine.mudlib` for `mudlib` lifecycle boundaries.

Instance And Transport Internals

`MudlibBoot` reads profile metadata, assembles runtime configuration, and loads startup content. `MudInstance` coordinates the running world, LPC objects, `Personas`, lifecycle dispatch, and scheduled work. `MudlibRouter` coordinates explicitly mounted worlds. `TelnetServer` owns the listener and connection path.

Follow an input from transport to instance to LPC command handling when debugging player behavior. Follow an `efun` from its catalog through runtime context and instance services when debugging host effects. `compiler.runtime` is generated-code support; `engine.world` owns the engine's world model.

`AdminClient` connects to the authenticated loopback `AdminServer`; each client gets an attached `AdminCli` command session. `InstanceHost.administer` serializes commands against the primary world with the same router/instance locks used by player dispatch and ticks. Admin socket reads and response writes occur outside those locks. The client never boots a runtime. Storage adapters live under `persistence`. Concurrency hardening and alignment between LPC location bookkeeping and engine containment remain tracked in `docs/DEFERRED_WORK.md`; avoid implying that package boundaries alone prove production isolation.

Testing And Verification

Run the repository baseline from its root:

```
mvn test
```

For a targeted change, select the relevant JUnit class, for example:

```
mvn -Dtest=AdminCliTest test
```

Some integration checks require an external database and configured password environment variable. A missing prerequisite is an incomplete integration check, not a passing compatibility result. Read the failing test and profile before classifying it as a Java regression.

Use `target/surefire-reports/` for test outcomes. Informational compatibility reports and full preload scans answer different questions from assertions and live player tests. Shared scan tooling is `scripts/scan-mudlib-preload-compile.sh`; profile wrappers remain under the owning `mudlib's jvmud/` directory.

For runtime changes, restart the affected profile and replay the relevant player route. For

documentation-only edits, regenerate outputs and check local links, anchors, commands against source, and PDF layout. See [Documentation Generation](#).

Troubleshooting

Begin with the smallest failing step: Java build, manifest validation, object load, listener startup, login, or command handling. Keep the first diagnostic and the exact manifest path; later errors may be consequences of the first failure.

Build Failures

- **Release 21 not supported:** compare `java -version` and `mvn -version`; Maven may be using a different JDK. Select the intended JDK before rebuilding.
- **Dependencies cannot resolve:** check access to the configured Maven repository. An offline cache must already contain the required artifacts.
- **Tests fail on database setup:** inspect the selected profile's external service requirements. Do not put a password in the manifest to silence the failure.
- **Permission denied launching a script:** use a POSIX shell environment and check the script's executable bit in the checkout.

Retain Maven's first error and the relevant Surefire report. Repeatedly rebuilding without changing the failing prerequisite is unlikely to help.

Object Load Failures

1. Confirm the source path is relative to the selected mudlib root, not the Java repository root.
2. Inspect `include_paths`, inherited source paths, and preload declarations.
3. In the admin shell, set `verbosity watch` and run `load <path>`.
4. Use `preprocess`, `scan`, `parse`, or `ir` to stop at the earliest failing stage.
5. Check explicit types, optional `language_features`, `efun` aliases, and capability grants before changing source semantics.

Unknown manifest keys fail fast. A missing `initial_place` or invalid profile mapping is a hosting configuration problem. Runtime invocation failures after a successful load need an execution test, not just another compile scan.

Login And Command Failures

Symptom	Check
Connection refused	Wait for the listening message; verify the process is still running on the configured address and player port (default localhost:4000).
Address already in use	Stop your earlier listener or choose another port with <code>--port 4001</code> .
Prompts repeat or login fails	Confirm the selected profile and account policy; inspect the server log and that profile's writable storage.
Admin command is unrecognized in Telnet	Use <code>scripts/jvmud-cli --port <admin-port></code> ; the server must have been started with <code>--admin-port</code> .
Movement or look is unrecognized	Complete login and read the active mudlib's help; verify command registration and lifecycle mapping.
GMCP output is absent	Check client negotiation, <code>session_control</code> , and the mudlib's package emission. GMCP is not required for ordinary text play.

For mounted-world issues, first prove each profile can boot directly, then test the mount configuration and transfer route. Capture the first failing transition.

Documentation Generation

Edit `manual/**/*.adoc`, not `docs/manual/index.html` or the PDF. From the repository root, generate both manual formats:

```
mvn -Pmanual-docs -DskipTests process-resources
```

The outputs are `docs/manual/index.html` and `docs/manual/jvmud-user-manual.pdf`. Shared HTML styling and navigation come from `manual/docinfo.html` and `manual/docinfo-header.html`; PDF typography and page flow come from `manual/pdf-theme.yml`.

Generate the Java API with:

```
mvn -Psite-docs -DskipTests javadoc:javadoc
python3 src/test/scripts/check-docs.py
```

Javadocs live under `docs/api/apidocs/`; `docs/api/index.html` is the authored package guide. The checker verifies local HTML links, fragment targets, and manual placeholders. It does not check external URLs or prove visual quality.

Preview the static site with `python3 -m http.server 8000 --directory docs` and open <http://localhost:8000/>. Stop that preview with Ctrl+C.

If generation fails, check Maven dependency resolution, missing AsciiDoc includes, and the first generator error. Regenerate HTML and PDF together, then inspect the PDF contents, page breaks,

and long code/table cells before publishing. See docs/README.md for source ownership and docs/DOCUMENTATION_GAPS.md for remaining coverage work.

Appendixes

Use these references while following the procedures. The manifest key inventory and command syntax describe the current implementation; compatibility limits are explicit rather than implied by a successful build.

Commands

These commands run inside `scripts/jvmud-cli --port <admin-port>`. Paths name source files; handles identify loaded objects. Use `objects` to find handles. Arguments to `call` are passed as strings; quote strings containing spaces. Numeric text is not automatically converted to LPC integers. `help` is the live syntax reference.

Command	Alias	Purpose
<code>help</code>	<code>h</code>	Show this command reference.
<code>call <handle> <method> [args...]</code>	—	Invoke a method on a loaded object handle.
<code>cat <path></code>	—	Print a file from the virtual mudlib filesystem.
<code>cd [path]</code>	—	Change the current virtual mudlib directory.
<code>clone <path></code>	—	Compile if needed and create a new LPC object instance.
<code>compile <path></code>	<code>c</code>	Compile an LPC source file to bytecode without loading it.
<code>destruct <handle></code>	<code>x</code>	Remove an object from the runtime.
<code>inspect <handle></code>	<code>i</code>	Show object state, inventory, environment, and methods.
<code>ir <path></code>	<code>ir</code>	Compile through lowering and print the typed IR.
<code>load <path></code>	<code>l</code>	Compile, load, and register an LPC object.
<code>look <handle></code>	<code>k</code>	Call <code>long()</code> or <code>short()</code> and display object text.
<code>ls [path]</code>	—	List a virtual mudlib directory or file.
<code>move <handle> <dest></code>	<code>m</code>	Move one object handle into another object's inventory.
<code>objects</code>	<code>o</code>	List registered live object handles.
<code>parse <path></code>	<code>p</code>	Preprocess, scan, parse, and print the LPC AST.
<code>preprocess <path></code>	<code>pp</code>	Expand includes, macros, and preprocessor directives.

Command	Alias	Purpose
pwd	—	Print the current virtual mudlib directory.
reload <path>	r	Recompile and replace a loaded LPC object.
scan <path>	s	Preprocess and print scanner tokens for an LPC source file.
verbosity [quiet normal watch]	v	Show or change compiler/shell output detail.
where <handle>	w	Show the environment containing an object.
quit	q	Disconnect this CLI session; the world keeps running.

The terminal launcher for play is `scripts/jvmud-start [--bind <address>] [--port <port>] [--admin-port <port>] [--admin-token-file <path>] [--trace-startup-loads] <mudlib-config-file>`. Inside a player client, profiles using the default transport prefix reserve `//help` and `//quit` for transport controls. The mudlib owns all gameplay verbs.

Configuration Reference

Manifests use `key = value`, one declaration per line. Lists are comma-separated. Blank lines and comments are ignored. Simple surrounding quotes are removed; this is not a general-purpose quoted-string format, and still starts a comment. Unknown keys are rejected by `MudlibBoundaryConfigReader`.

Use a checked-in profile as the complete example. The following groups cover the reader's exact keys and supported key prefixes. Optional values can still become required by the selected boot, session, or database policy.

Identity and paths

Key	Meaning
game_id, game_name	Stable profile identity and display name.
mudlib_root	LPC source root; relative paths resolve from the manifest directory.
initial_place	Startup Place object path, required for hosted boot.
persona_object, player_object	Interactive object path; <code>persona_object</code> takes precedence over the legacy spelling when both are present.
mudlib_object	Optional boundary-declaration LPC object.
mfun_object	Mudlib-global function object.
include_paths	Comma-separated source include search paths.
preload_file	Profile preload-list file.
preload_objects	Additional comma-separated startup object paths.

Key	Meaning
<code>mount.<game-id></code>	Mounted profile config, relative to this manifest's directory.

Use mudlib-rooted object paths such as `room/church`; Java package paths are not LPC paths. The launch scripts resolve a relative manifest argument from the repository root. `mudlib_root` resolves from the manifest, a different base.

Sessions and presentation

Key	Meaning
<code>session_policy</code>	Optional host-managed login policy. <code>filesystem_accounts</code> selects reusable local account persistence; omit for profile-owned login.
<code>player_prompt</code> , <code>command_prompt</code>	Prompt text; <code>player_prompt</code> takes precedence.
<code>presentation.connected_banner</code>	Text shown on connection.
<code>transport.control_prefix</code>	Escape prefix for transport controls; default <code>//</code> .
<code>diagnostics.location_command</code>	Optional profile-selected location diagnostic command.
<code>max_line_length</code>	Presentation line width, validated against engine bounds.
<code>show_ruler</code>	Presentation ruler toggle. Booleans accept <code>true/false</code> , <code>yes/no</code> , <code>on/off</code> , or <code>1/0</code> .

Host and port are not manifest keys. The shared Telnet launcher uses loopback port 4000; passing a port as a second positional argument fails.

Language and host access

Key	Meaning
<code>language_features</code>	Optional syntax gates: <code>protected_evaluation</code> , <code>typed_function_literals</code> , <code>inline_callables</code> , <code>multi_value_mappings</code> , <code>varargs</code> .
<code>compiler.dynamic_types</code>	Optional dynamic value typing, default <code>false</code> . Treat declared fields, locals, parameters, and returns as stable mixed values. Cannot be combined with <code>field/local</code> type overrides.
<code>transpilation.untyped_methods</code>	Supply missing method return and parameter types in memory; default <code>false</code> and independent of dynamic typing.
<code>transpilation.implicit_self_calls</code>	Resolve unknown bare calls on the current object at runtime; default <code>false</code> .
<code>transpilation.overrides</code>	Manifest-relative JSON file containing checked field, local, or method-varargs adaptations. LP245 retains only a method-varargs rule.

Key	Meaning
<code>engine_capabilities</code>	Capability grants: <code>mudlib_files</code> , <code>database</code> , <code>session_control</code> , <code>host_control</code> .
<code>engine_function.<native-name></code>	One or more mudlib aliases for a native engine function.
<code>compatibility_object</code>	Explicit compatibility-global function object.
<code>compatibility_overrides</code>	Names allowed to override mudlib-global functions.
<code>compatibility.predefine.<macro></code>	Object-like compatibility macro replacement.
<code>compatibility.function_predefine.<macro>.<argument></code>	Single-argument function-like compatibility macro replacement.

Feature flags enable syntax; capabilities enable restricted host operations. Start with the active profile's actual requirements. A compatibility alias does not bypass a capability grant.

Lifecycle and time

Key	Meaning
<code>handled_lifecycle_events</code>	Comma-separated native event declarations.
<code>lifecycle.<event></code>	Map a native lifecycle event to a mudlib method. Event names are normalized for case, spaces, and hyphens.
<code>temporal_tick_method</code>	Mudlib method for scheduler-backed recurring behavior.
<code>temporal_tick_interval</code>	Interval in seconds by default; also accepts <code>s</code> , <code>seconds</code> , <code>ms</code> , or <code>milliseconds</code> . Negative values fail. Zero does not install an interval override.

Refer to [lifecycle events](#) for event roles and verify method signatures against the selected profile.

Database

Key	Meaning
<code>database.url</code> , <code>database.jdbc_url</code>	JDBC URL; <code>database.url</code> takes precedence.
<code>database.user</code>	Database user for the selected profile.
<code>database.password_env</code>	Name of a required nonblank environment variable containing the password. Preferred for configured profiles.
<code>database.password</code>	Literal password alternative supported by the reader; do not commit secrets. Mutually exclusive with <code>database.password_env</code> .

A missing named password environment variable fails manifest loading. Setting the variable does not provision a database, create a schema, or grant database access. Those are profile-specific prerequisites.

Compatibility Notes

LPC dialects differ. Accepted syntax, successful compilation, object loading, and correct interactive behavior are separate levels of support. Optional language features are selected per profile; legacy efun names and lifecycle methods are boundary mappings.

`docs/COMPATIBILITY_FEATURE_PROVENANCE.md` records why compatibility features exist.

`docs/DEFERRED_WORK.md` records implementation debt; `docs/FUTURE_DESIGN_IDEAS.md` collects proposals. None is a substitute for the active source and tests, or a guarantee of full legacy-driver parity.

Database provisioning and recovery remain profile-specific. Start with the credential-free examples in this manual before taking on that environment.

Glossary

Use [the JVMud Glossary](#) for the full conceptual vocabulary. Its controlling repository source is `docs/GLOSSARY.md`; the controlling design principles are in `docs/PRINCIPLES.md`. The short terms section at the beginning of this manual defines the words needed for its procedures.